

Composição em Orientação a Objetos, Acoplamento e Coesão

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Revisão: Conceitos de Orientação a Objetos
- Composição
- Coesão
- Tipos de Coesão (visão conceitual)
- Relação com o Princípio da Responsabilidade Única (SRP)
- Exercício: Avaliando a Coesão de Classes
- Acoplamento
- Tipos de Acoplamento
- Formas Comuns de Alto Acoplamento
- Estratégias para Reduzir Acoplamento
- Relação entre os Conceitos

Revisão: Conceitos de Orientação a Objetos

- **Classes**

- Classe: é o molde ou modelo que define como os objetos de um tipo serão estruturados.
 - Contém atributos (dados, estado) e métodos (funções, comportamentos).

```
class ContaBancaria:
    def __init__(self, titular: str, saldo: float):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor: float):
        self.saldo += valor

    def sacar(self, valor: float):
        self.saldo -= valor
```

Revisão: Conceitos de Orientação a Objetos

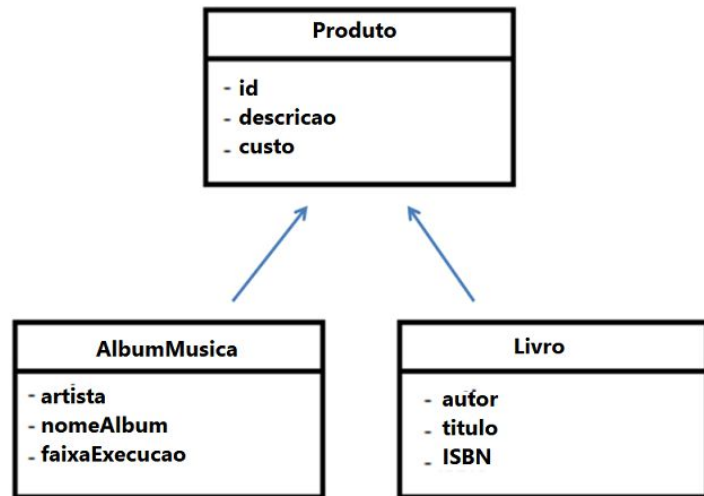
- **Objeto:** é uma instância concreta de uma classe — algo real que existe na execução do programa.
- Exemplo:

```
# Criação de uma instância da classe
conta_joao = ContaBancaria("João", 1000.0)
conta_joao.depositar(500.0)
```

Classe = definição (molde).
Objeto = instância (ocorrência real dessa definição).

Revisão: Conceitos de Orientação a Objetos

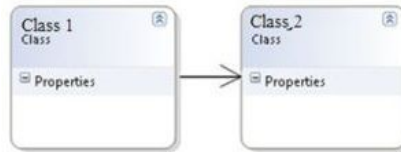
- **Herança:** É um relacionamento do tipo “é um” (is a), onde uma classe derivada herda atributos e métodos de uma classe base, reutilizando código e especializando comportamentos.
 - A classe derivada representa um subtipo da classe base, podendo adicionar ou sobrescrever membros.
 - Exemplo
 - Uma classe ProdutoEletronico herda de Produto, pois um produto eletrônico é um produto.



Relacionamentos entre Classes

- **Associação**

- Representa uma relação simples de uso entre classes.
- Um objeto conhece ou usa outro, mas não o “possui”.
- Exemplo:
 - Um Professor ministra aulas em um Curso.



Associação

```
class Professor:
    def __init__(self, nome: str):
        self.nome = nome

class Curso:
    def __init__(self, nome: str,
professor: Professor):
        self.nome = nome
        self.professor = professor
```

Relacionamentos entre Classes

- **Agregação**

- Representa uma relação “tem-um”, mas com independência entre as partes.
- O objeto “todo” agrega partes que podem existir separadamente.
- Exemplo:
 - Uma Turma contém Alunos, mas os alunos existem mesmo sem a turma.



```
class Aluno:
    def __init__(self, nome:
str):
        self.nome = nome

class Turma:
    def __init__(self, codigo:
str, alunos: list[Aluno]):
        self.codigo = codigo
        self.alunos = alunos
```

Relacionamentos entre Classes

- **Composição**

- Representa uma relação “todo-parte” forte:
 - o objeto “todo” controla o ciclo de vida das partes.
- Se o todo deixar de existir, as partes também deixam.
- Exemplo:
 - Um Pedido contém ItensPedido; se o pedido é excluído, os itens também são.

Composição

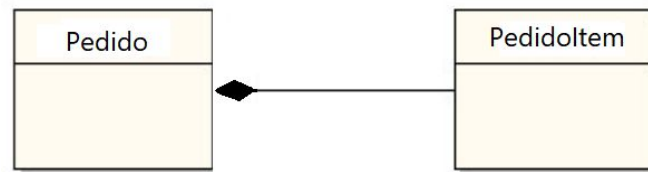
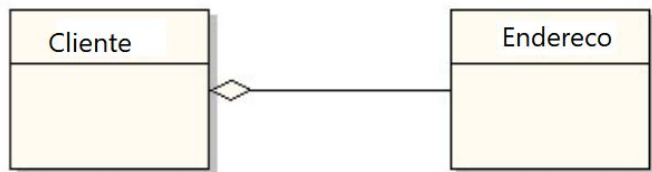
- Definição: tipo de associação forte onde um objeto “é dono” de outro.
- Quando o objeto dono é destruído, o objeto contido também é.
- Sintaxe: representada por “tem-um” (“has-a”).

```
class Motor:
    def ligar(self):
        print("Motor ligado")

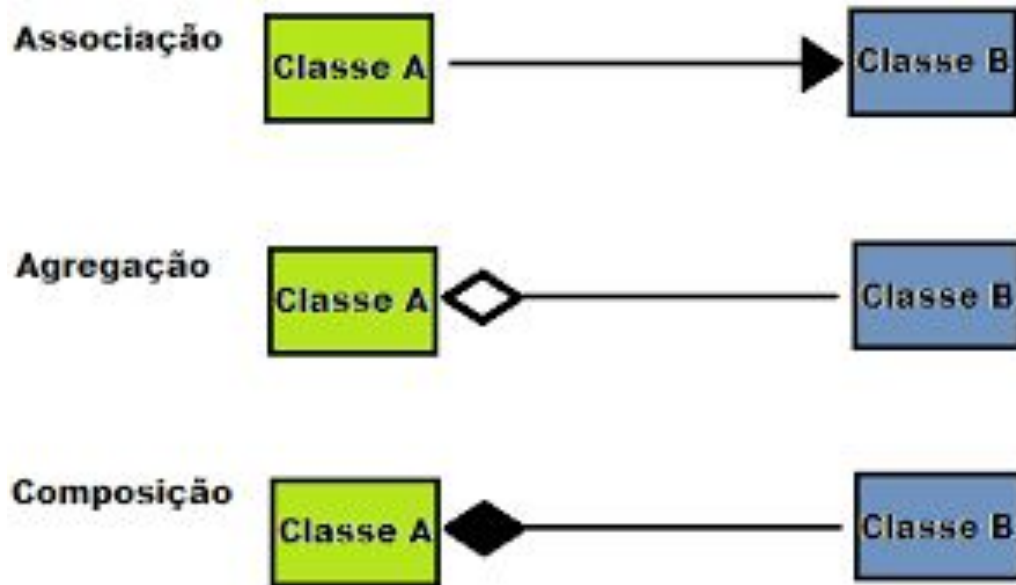
class Carro:
    def __init__(self):
        self.motor = Motor() # composição
    def ligar(self):
        self.motor.ligar()
        print("Carro ligado")
```

Diferença entre Agregação e Composição

- Agregação: o objeto pode existir sozinho.
- Composição: o objeto não existe sem o outro.



Diferença entre Agregação e Composição



Coesão

- **Definição**
 - Coesão é uma medida de quão fortemente os elementos internos de uma classe (atributos e métodos) estão relacionados entre si e contribuem para uma única responsabilidade.
 - Em outras palavras, uma classe coesa tem um único propósito claro — todos os seus métodos e atributos colaboram para cumprir a mesma função ou objetivo.
- Quanto maior a coesão, mais fácil é entender, manter e reutilizar o código.
- Quanto menor a coesão, mais provável é que a classe viole o Princípio da Responsabilidade Única (SRP) da Programação Orientada a Objetos.

Tipos de Coesão (visão conceitual)

- Embora o termo venha da engenharia de software clássica, podemos simplificar os níveis de coesão em três categorias para OO:
- **Alta coesão** → a classe tem um foco bem definido e claro.
- **Média coesão** → a classe realiza tarefas relacionadas, mas poderia ser dividida em partes menores.
- **Baixa coesão** → a classe realiza muitas tarefas diferentes, sem relação direta entre elas.

Alta coesão

- A classe tem uma responsabilidade clara e única.
- A classe trata apenas de operações matemáticas. Todas as funções estão diretamente relacionadas ao mesmo propósito.

```
class Calculadora:
    def somar(self, a, b):
        return a + b

    def subtrair(self, a, b):
        return a - b
```

Média coesão

- A classe ainda é coerente, mas poderia ser dividida em partes mais específicas.
- Todas as funções se relacionam ao processo de um pedido, mas gerar nota fiscal e enviar e-mail poderiam estar em classes separadas (ServicoNotaFiscal, ServicoEmail).

```
class Pedido:
    def calcular_total(self):
        pass

    def gerar_nota_fiscal(self):
        pass

    def enviar_email_confirmacao(self):
        pass
```

Baixa coesão

- A classe mistura várias responsabilidades sem relação direta.
- Essa classe faz de tudo — cadastro, relatório, notificação, backup — funções que pertencem a contextos diferentes. É difícil manter e reutilizar.

```
class Sistema:
    def cadastrar_usuario(self):
        pass

    def gerar_relatorio_financeiro(self):
        pass

    def enviar_notificacao(self):
        pass

    def fazer_backup(self):
        pass
```

Níveis de Coesão em Orientação a Objetos

Nível	Descrição	Exemplo resumido	Comentário
Alta coesão	A classe tem uma responsabilidade clara e única.	Uma classe Calculadora que apenas realiza operações matemáticas como somar e subtrair.	Foco único e bem definido. Fácil de entender, testar e manter.
Média coesão	A classe faz tarefas relacionadas, mas poderia ser dividida.	Uma classe Pedido que calcula o total, gera nota fiscal e envia e-mail de confirmação.	Tudo relacionado ao pedido, mas algumas funções poderiam ser separadas em outras classes.
Baixa coesão	A classe faz muitas coisas sem relação direta.	Uma classe Sistema que cadastra usuários, gera relatórios, envia notificações e faz backup.	Mistura várias responsabilidades. Torna o código complexo e difícil de manter.

Por que a Coesão é Importante

- **Melhora a legibilidade:** facilita o entendimento do que a classe faz.
- **Facilita manutenção:** erros e mudanças ficam localizados.
- **Aumenta a reutilização:** classes com propósito claro são mais fáceis de reaproveitar.
- **Reduz o acoplamento:** quando cada classe cuida do seu papel, depende menos das outras.

“Se você não consegue descrever o que uma classe faz em uma única frase, ela provavelmente tem baixa coesão.”

Relação com o Princípio da Responsabilidade Única (SRP)

- A coesão está intimamente relacionada ao S do acrônimo SOLID, ou seja, ao SRP (Single Responsibility Principle) — Princípio da Responsabilidade Única.
- **Definição do SRP**
 - O SRP afirma que: “Uma classe deve ter apenas um motivo para mudar.” Em outras palavras, uma classe deve ter uma única responsabilidade bem definida, focando em um único papel dentro do sistema.

Ligação com a Coesão

- Uma classe coesa tende naturalmente a respeitar o SRP, pois concentra todas as suas operações em torno de um mesmo propósito.
- Quando a coesão é baixa, isso significa que a classe possui múltiplas responsabilidades — logo, vários motivos para mudar, violando o SRP.
- Assim, a coesão é um indicador prático do quanto uma classe está em conformidade com o SRP.

Exercício: Avaliando a Coesão de Classes

- Observe o código de cada classe a seguir e responda:
- O nível de coesão da classe (Alta, Média ou Baixa).
- Justifique sua resposta em uma frase.

Classe Aluno

```
class Aluno:
    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula

    def calcular_media(self, notas):
        return sum(notas) / len(notas)

    def verificar_aprovacao(self, notas):
        return self.calcular_media(notas) >= 7
```

Pergunta: Essa classe apresenta alta, média ou baixa coesão?

Classe Aluno

- Nível de Coesão: Alta
- Justificativa: Todos os métodos e atributos estão relacionados a um mesmo propósito: representar o desempenho e status de aprovação de um aluno. Nenhum método foge da responsabilidade central da classe.
- É um ótimo exemplo de classe com uma única responsabilidade bem definida — fácil de entender e manter.

Classe Biblioteca

```
class Biblioteca:  
    def adicionar_livro(self, livro): ...  
    def emprestar_livro(self, livro, usuario): ...  
    def enviar_email_atraso(self, usuario): ...  
    def gerar_relatorio_financeiro(self): ...
```

Pergunta: Essa classe é coesa?

Classe Biblioteca

- Nível de Coesão: Baixa
- Justificativa: A classe mistura várias responsabilidades distintas: controle de acervo, comunicação com usuário e relatórios financeiros. Essas funções pertencem a domínios diferentes e deveriam ser separadas.
- Excelente para mostrar o perigo da “classe Deus” (God Class) — uma classe que tenta fazer tudo e acaba difícil de testar e evoluir.

Classe ContaBancaria

```
class ContaBancaria:
    def __init__(self, titular, saldo):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor):
        self.saldo += valor

    def sacar(self, valor):
        if valor <= self.saldo:
            self.saldo -= valor
            return True
        return False

    def exibir_saldo(self):
        print(f"Saldo atual: R$ {self.saldo}")
```

Pergunta: O nível de coesão é alto, médio ou baixo?

Classe ContaBancaria

- Nível de Coesão: Alta
- Justificativa: Todos os métodos lidam diretamente com o saldo e as operações bancárias da conta. O foco da classe é único e bem definido.
- Mostra como coesão alta leva à clareza de propósito — qualquer pessoa sabe o que essa classe faz apenas lendo o nome.

Classe Aplicativo

```
class Aplicativo:
    def __init__(self):
        self.usuario = None
        self.midia = None
        self.versao = "1.0"

    def login(self, usuario):
        self.usuario = usuario
        print(f"{usuario} logado com sucesso.")

    def reproduzir(self, arquivo):
        print(f"Reproduzindo {arquivo}...")

    def atualizar(self):
        self.versao = "1.1"
        print(f"Aplicativo atualizado para versão {self.versao}.")
```

Pergunta: O nível de coesão é alto, médio ou baixo?

Classe Aplicativo

- Nível de Coesão: Média a Baixa
- Justificativa: Há funções de autenticação, execução de mídia e manutenção no mesmo lugar. Embora possam coexistir em um app real, essas tarefas pertencem a módulos distintos.
- Bom para mostrar que coesão média ocorre quando o escopo ainda não está totalmente errado, mas pode ser refatorado em classes específicas (LoginService, Player, Atualizador etc.).

Classe PedidoOnline

```
class PedidoOnline:  
    def adicionar_produto(self, produto): ...  
    def calcular_total(self): ...  
    def aplicar_desconto(self, cupom): ...  
    def gerar_pdf_pedido(self): ...
```

Pergunta: O nível de coesão é alto, médio ou baixo?

Classe PedidoOnline

- Nível de Coesão: Média
- Justificativa: Os três primeiros métodos tratam da lógica de negócio (pedido e preço), mas o último (gerar_pdf_pedido) trata de apresentação/formatação, não da regra do pedido.
- Use esse exemplo para discutir refatoração: a geração de PDF poderia ser movida para uma classe RelatorioPedido, mantendo PedidoOnline focada apenas no domínio de pedidos.

Acoplamento

- **Definição**
 - Acoplamento mede o grau de dependência entre classes ou módulos de um sistema. Em outras palavras, indica quanto uma classe precisa conhecer ou depender de outras para funcionar.
- Quanto maior o acoplamento, mais as classes estão interligadas — o que pode gerar fragilidade.
- Quanto menor o acoplamento, mais independente, reutilizável e fácil de manter o sistema se torna.

Coesão x Acoplamento

- Esses dois conceitos são complementares:

Conceito	O que mede	Ideal
Coesão	Relação entre os elementos <i>dentro</i> de uma classe	Alta
Acoplamento	Relação de <i>dependência</i> entre classes diferentes	Baixo

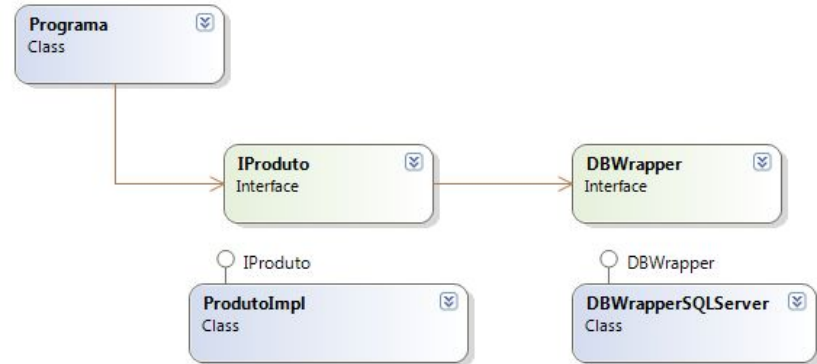
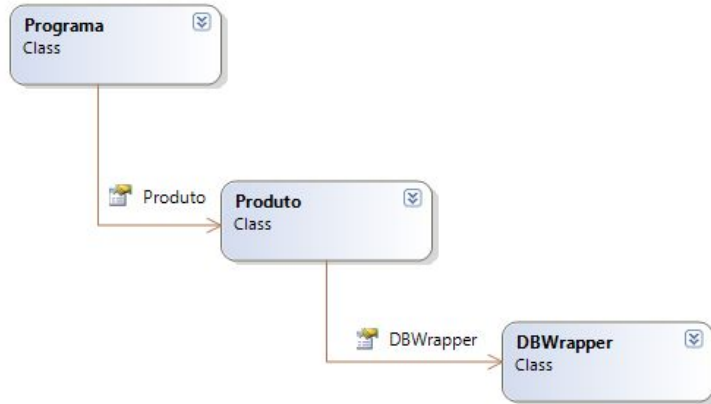


Regra de ouro:

“Classes devem ser altamente coesas e fracamente acopladas.”

Tipos de Acoplamento

- Alto Acoplamento
- Baixo Acoplamento



Alto Acoplamento

- A classe depende fortemente de detalhes de outra classe.
- Mudanças em uma impactam diretamente a outra.
- Dificulta manutenção, testes e reuso.

Exemplo:

```
class Pedido:
    def __init__(self):
        self.pagamento = Pagamento() #
depende diretamente da classe Pagamento

    def processar(self):
        self.pagamento.confirmar() #
chamada direta
```

Se a classe Pagamento mudar (nome de método, parâmetros etc.), Pedido também precisará mudar. Há **dependência rígida** entre elas.

Baixo Acoplamento

- A classe depende apenas de interfaces, abstrações ou parâmetros externos.
- Mudanças em uma classe têm pouco ou nenhum impacto nas outras.
- Facilita testes unitários, substituição de componentes e injeção de dependências.

Exemplo (injeção de dependência):

```
class Pedido:
    def __init__(self, pagamento):
        self.pagamento = pagamento #
recebe a dependência de fora

    def processar(self):
        self.pagamento.confirmar()
```

Pedido não cria o Pagamento por conta própria — ele recebe um objeto de pagamento já pronto. Assim, pode funcionar com qualquer tipo de pagamento (cartão, pix, boleto etc.) sem precisar ser alterado.

Formas Comuns de Alto Acoplamento

- Instanciar dependências diretamente dentro da classe (`self.x = OutraClasse()`);
- Acessar diretamente atributos privados de outra classe;
- Chamar muitos métodos encadeados (ex: `obj.a().b().c()`);
- Depender de classes concretas, e não de abstrações.

Estratégias para Reduzir Acoplamento

- Injeção de dependência → passar as dependências como parâmetros (construtor ou método).
- Interfaces/Abstrações → depender de um “contrato” e não de uma implementação.
- Design patterns como Strategy, Observer e Factory ajudam a isolar dependências.
- Princípio da Inversão de Dependência (D do SOLID):
 - Módulos de alto nível não devem depender de módulos de baixo nível, ambos devem depender de abstrações.

Exemplo Comparativo

```
class Notificador:
    def enviar_email(self, mensagem):
        print(f"Email enviado: {mensagem}")

class Usuario:
    def __init__(self):
        self.notificador = Notificador() # forte dependência

    def cadastrar(self, nome):
        print(f"Usuário {nome} cadastrado.")
        self.notificador.enviar_email("Bem-vindo!")
```

✗ Alto acoplamento

Exemplo Comparativo

```
class Notificador:
    def enviar(self, mensagem):
        print(f"Mensagem enviada: {mensagem}")

class Usuario:
    def __init__(self, notificador):
        self.notificador = notificador # dependência injetada

    def cadastrar(self, nome):
        print(f"Usuário {nome} cadastrado.")
        self.notificador.enviar("Bem-vindo!")
```

Aqui, Usuario não depende do *tipo* de notificação (pode ser e-mail, SMS, push...). Ele apenas usa uma interface genérica — reduzindo o acoplamento e aumentando a flexibilidade.

✓ Baixo acoplamento

Benefícios do Baixo Acoplamento

- Facilita manutenção e evolução do código.
- Permite testes isolados com mocks e stubs.
- Aumenta a reutilização de classes em outros contextos.
- Melhora a modularidade e substituição de implementações sem impacto global.

“Se uma mudança em uma classe obriga você a alterar várias outras, o acoplamento está muito alto.”

Resumo

Conceito	Descrição	Exemplo
Alto acoplamento	Classes fortemente dependentes umas das outras	Pedido cria Pagamento() internamente
Baixo acoplamento	Classes independentes, comunicam-se por interfaces ou parâmetros	Pedido recebe Pagamento no construtor

Relação entre os Conceitos

Conceito	Desejável	Objetivo
Coesão	Alta	Classes focadas e simples
Acoplamento	Baixo	Facilitar manutenção e testes
Composição	Forte relação controlada	Melhor modelagem e encapsulamento

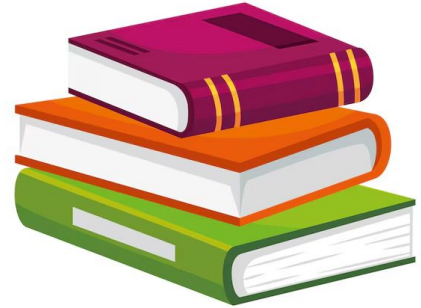
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

